If the downstream service belongs to your system, you might use a rate limiter to avoid causing DoS problems for yourself. In this case, though, a *bulkhead* pattern (or *concurrent request limiter*) would be a better fit, setting constraints on how many concurrent requests are allowed and queuing up the blocked ones. Even better is an adaptive bulkhead, for which the concurrency limits are dynamically updated by an algorithm to better adapt to the elasticity of cloud infrastructure.

Server-side rate limiters are for constraining the number of requests received by an upstream service (or client) in a given period. This pattern is handy when implemented in an API gateway to protect the whole system from overloading or from DoS attacks. When the number of users increases, the system should scale in a resilient way, ensuring an acceptable quality of service for all users. Sudden increases in user traffic are expected, and they are usually initially addressed by adding more resources to the infrastructure or more application instances. Over time, though, they can become a problem and even lead to service outages. Server-side rate limiters help with that.

When a user has exceeded the number of allowed requests in a specific time window, all the extra requests are rejected with an `HTTP 429 - Too Many Requests` status. The limit is applied according to a given strategy. For example, you can limit requests per session, per IP address, per user, or per tenant. The overall goal is to keep the system available for all users in case of adversity. That is the definition of resilience. This pattern is also handy for offering services to users depending on their subscription tiers. For example, you might define different rate limits for basic, premium, and enterprise users.

Resilience4J supports the client-side rate limiter and bulkhead patterns for both reactive and non-reactive applications. Spring Cloud Gateway supports